

---

# **Software Requirements Specification**

**for**

# **Authentication Library**

**Version 1.0 approved**

**Prepared by Tunahan Gokgoz**

**Università degli Studi di Firenze**

**20 April 2026**

## Table of Contents

|                                                    |                              |
|----------------------------------------------------|------------------------------|
| <b>1. Introduction .....</b>                       | <b>1</b>                     |
| 1.1 Purpose .....                                  | 1                            |
| 1.2 Document Conventions .....                     | 1                            |
| 1.3 Intended Audience and Reading Suggestions..... | 1                            |
| 1.4 Product Scope .....                            | 1                            |
| 1.5 References.....                                | 2                            |
| <b>2. Overall Description.....</b>                 | <b>2</b>                     |
| 2.1 Product Perspective .....                      | 2                            |
| 2.2 Product Functions .....                        | 2                            |
| 2.3 User Classes and Characteristics.....          | 2                            |
| 2.4 Operating Environment.....                     | 3                            |
| 2.5 Design and Implementation Constraints .....    | 3                            |
| 2.6 User Documentation .....                       | 3                            |
| 2.7 Assumptions and Dependencies.....              | 3                            |
| <b>3. External Interface Requirements .....</b>    | <b>3</b>                     |
| 3.1 User Interfaces.....                           | 3                            |
| 3.2 Hardware Interfaces.....                       | 3                            |
| 3.3 Software Interfaces.....                       | 4                            |
| 3.4 Communications Interfaces.....                 | 4                            |
| <b>4. System Features.....</b>                     | <b>4</b>                     |
| 4.1 System Feature 1 .....                         | Error! Bookmark not defined. |
| 4.2 System Feature 2 (and so on).....              | Error! Bookmark not defined. |
| <b>5. Other Nonfunctional Requirements .....</b>   | <b>8</b>                     |
| 5.1 Performance Requirements .....                 | 8                            |
| 5.2 Safety Requirements .....                      | 8                            |
| 5.3 Security Requirements.....                     | 8                            |
| 5.4 Software Quality Attributes .....              | 8                            |
| 5.5 Business Rules.....                            | 9                            |
| <b>6. Other Requirements .....</b>                 | <b>9</b>                     |
| <b>7. Traceability matrix.....</b>                 | <b>9</b>                     |

## Revision History

| Name           | Date          | Reason For Changes | Version |
|----------------|---------------|--------------------|---------|
| Tunahan Gokgoz | 20 April 2026 | Initial draft      | 1.0     |
|                |               |                    |         |

# 1. Introduction

## 1.1 Purpose

This Software Requirements Specification (SRS) describes the functional and non-functional requirements of the Authentication Library, version 1.0. The Authentication Library is a small Java component that handles user registration, credential verification, account modification, and basic protection against brute-force login attempts. This document covers the full scope of the library, which is delivered as a standalone Java module intended to be reused by other applications that need a simple authentication layer.

## 1.2 Document Conventions

Requirement statements in this document follow the keyword conventions defined in **RFC 2119** ("Key words for use in RFCs to Indicate Requirement Levels"):

**MUST / SHALL** — an absolute requirement.

**MUST NOT / SHALL NOT** — an absolute prohibition.

**SHOULD** — a strong recommendation; deviation requires justification.

**MAY** — an optional item.

## 1.3 Intended Audience and Reading Suggestions

This document is intended for the following audiences:

**The course instructor (Prof. Andrea Ceccarelli)** as the reviewer of this academic deliverable.

## 1.4 Product Scope

The Authentication Library is a Java module that provides the minimum set of operations needed to manage user credentials and verify authentication attempts. The library was re-engineered from an original C implementation provided as the course running project, with the goal of producing a cleaner, object-oriented, and more maintainable version.

The library supports the following business goals:

- Allow an application to register new users with a username and password.
- Verify that a user attempting to authenticate provides correct credentials.
- Allow administrative operations on user records (deletion, password change).
- Provide a minimal defense against brute-force attacks through an account lockout mechanism.

The library is **not** a full identity-management platform.

## 1.5 References

- Sommerville, *Software Engineering*, 10th Edition, Pearson, 2015 — Chapter 4 (Requirements Engineering).
- S. Bradner, **RFC 2119** — *Key words for use in RFCs to Indicate Requirement Levels*, IETF, March 1997. <https://www.rfc-editor.org/rfc/rfc2119>
- Ceccarelli, *Ingegneria del Software* course slides, University of Florence, A.A. 2025–2026.
- *Authentication Library* — *original C implementation* (running project), <https://github.com/AndreaCecca/IS>
- Hands-on #1 deliverable: Agile methodology report and Java re-engineered implementation (this project, folder `hands_on1`).

## 2. Overall Description

### 2.1 Product Perspective

The Authentication Library is a self-contained Java component developed as part of the *Ingegneria del Software* course running project. It originates from a small C library provided by the instructor and was re-engineered into Java during Hands-on #1, with the goal of producing a more readable, modular, and maintainable codebase while preserving the original functional contract.

### 2.2 Product Functions

At a high level, the library performs four groups of operations: registering a new user, verifying the credentials of a user attempting to authenticate (with an account lockout after repeated failures), deleting an existing user, and changing the password of an existing user. An initialization function is also provided to set the path of the credential file. These functions are detailed individually in Section 4 (System Features).

### 2.3 User Classes and Characteristics

The library distinguishes two user classes. The first class is the **End User**, that is, the person whose credentials are stored in the library and who interacts with it indirectly through a host application to log in or change their own password.

The second class is the **Administrator**, who has the authority to register new users and delete existing user records. The administrator is assumed to be a trusted operator with privileged access to the host application or to a maintenance script that invokes the library directly.

## 2.4 Operating Environment

The library targets any environment running a Java Runtime Environment compatible with Java 17 or later.

## 2.5 Design and Implementation Constraints

The implementation language MUST be Java, in line with the goal of Hands-on #1 (re-engineering from C). No external libraries are used beyond the standard JDK, in order to keep the artifact minimal and easy to reuse. The credential storage format is a flat text file with one `username:password` record per line; this is a deliberate simplification consistent with the original C implementation and is sufficient for the academic scope of the project. The library is an academic exercise; it MUST NOT be used as-is in any production system.

## 2.6 User Documentation

The library is delivered with the following documentation: this SRS document, the agile-process report produced during Hands-on #1, basic Javadoc comments inside the source files, and a short `README.md` at the root of the GitHub repository that explains how to build and run the example client (`Main.java`).

## 2.7 Assumptions and Dependencies

This SRS assumes that the host application provides any user interface, input validation beyond the basic checks performed by the library, and the enforcement of administrative privileges. It also assumes that the credential file is stored on a file system to which only authorized processes have access, since the file contains plain-text passwords. The only external dependency of the library is the Java Standard Edition runtime; no third-party libraries are required to compile or execute the code.

# 3. External Interface Requirements

## 3.1 User Interfaces

The Authentication Library does not provide any graphical or textual user interface of its own. The only interface it exposes is a programmatic Java API.

## 3.2 Hardware Interfaces

The library has no specific hardware interface requirements. It runs on any machine capable of executing a Java Runtime Environment.

### 3.3 Software Interfaces

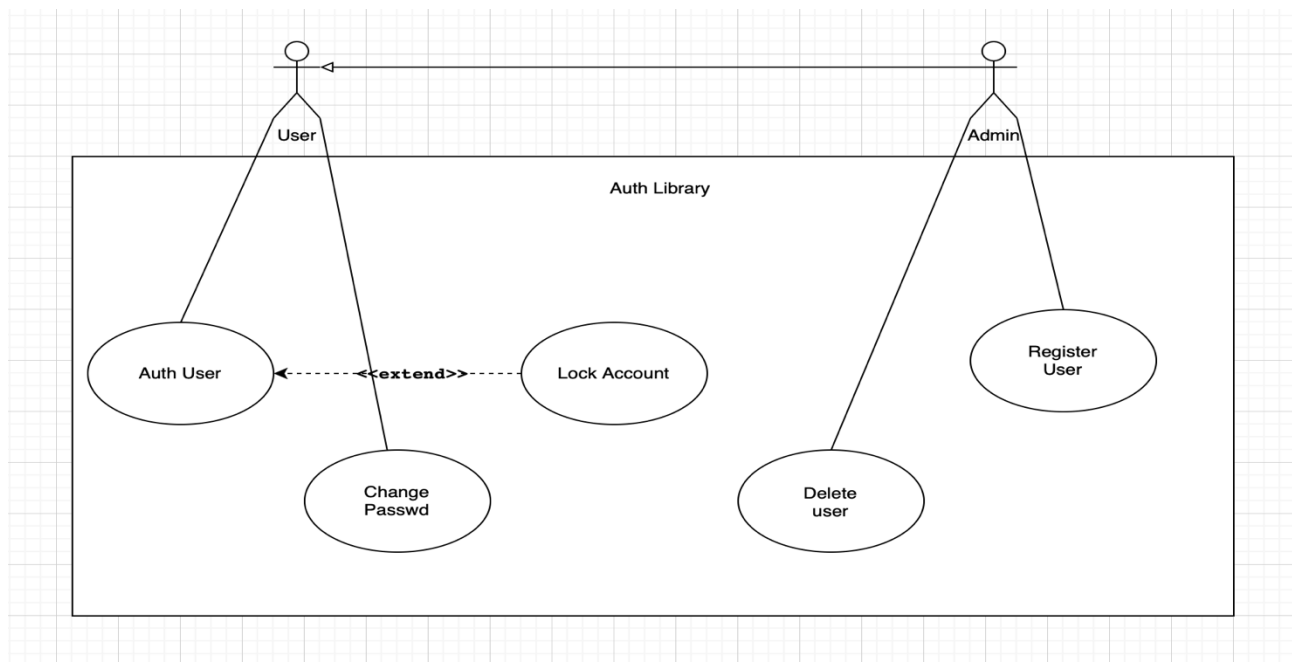
The library interfaces with two software components. The first is the **Java Standard Edition runtime**, version 17 or later, which provides the I/O and collection classes used by the implementation. The second is the **local file system**, accessed through the `java.io` package.

### 3.4 Communications Interfaces

Not applicable.

## 4. System Features

This section describes the functional requirements of the Authentication Library, organized by system feature. Four features have been identified, each corresponding to one public operation of the `AuthLibrary` class and to one or more Product Backlog Items from Hands-on #1. The use case diagram below summarizes the actors, use cases, and their relationships.



### 4.1 User Registration

**Objective** — Allow an administrator to register a new user in the library so that the user can later authenticate. The source of this requirement is REQ1 of the running project ("The Authentication Library must allow registering, modifying, removing users") and Product Backlog Item PBI-02 from Hands-on #1.

**Actors** — Administrator (primary, active).

## Flow of Events

*Basic Flow.* The administrator invokes the registration operation with a username and a password. The library validates that both inputs are non-null and non-empty, that neither contains a newline character, and that the username does not contain a colon. The library then verifies that no user with the same username already exists. If all checks pass, the new record is appended to the credential file and the operation returns success.

*Alternative Flow(s).* If any input is null or empty, if the username contains a colon, if either input contains a newline, or if the username already exists, the operation terminates immediately and returns a failure code without modifying the credential file.

### 4.1.1 Description and Priority

User registration is the entry point for every other operation in the system.

**Priority: High.** Without this feature, no other feature is meaningful.

### 4.1.2 Stimulus/Response Sequences

The administrator provides a new username and password. The system responds with a success code (0) when the record is stored, or a failure code (-1) when validation fails or the user already exists.

### 4.1.3 Functional Requirements

**REQ-SW-1:** The library **MUST** provide an operation that accepts a username and a password and persists the pair as a new user record.

**REQ-SW-2:** The library **MUST** reject any registration attempt in which the username or the password is null or empty.

**REQ-SW-3:** The library **MUST** reject any registration attempt in which the username contains the colon character (:), since the colon is the record separator in the storage format.

**REQ-SW-4:** The library **MUST** reject any registration attempt in which the username or the password contains a newline character.

**REQ-SW-5:** The library **MUST** reject any registration attempt for a username that is already present in the credential file.

**REQ-SW-6:** On successful registration, the library **MUST** append the new record to the credential file without modifying or removing existing records.

## 4.2 User Authentication

**Objective** — Verify that a person attempting to log in is presenting credentials that match a record previously stored by the library. This is the core of REQ2 of the running project ("The Authentication Library must verify that a user is requesting a legit authentication") and corresponds to PBI-01 and PBI-06 from Hands-on #1.

**Actors** — User (primary, active). Administrator inherits this capability through actor generalization.

## Flow of Events

*Basic Flow.* The user provides a username and a password. The library first checks whether the account is currently locked due to previous failed attempts. If it is not locked, the library searches the credential file for a record matching the given username and compares the stored

password with the one provided. On a successful match, the failed-attempts counter for that username is reset to zero and a success code is returned.

*Alternative Flow(s).* If the credentials do not match, the failed-attempts counter for that username is incremented and a failure code is returned. If the counter reaches the configured threshold (three attempts), the account is locked and any subsequent authentication attempt is rejected with a dedicated "locked" code without consulting the credential file. If either input is null, the operation returns an input-error code immediately.

#### 4.2.1 Description and Priority

User authentication is the primary security-relevant operation of the library and the most frequently invoked in any realistic usage. **Priority: High.**

#### 4.2.2 Stimulus/Response Sequences

The user submits a username and a password. The system responds with a code that distinguishes four outcomes: authentication succeeded, credentials did not match, account is locked, or input was invalid. The exact response depends on the current state of the failed-attempts counter for that username.

#### 4.2.3 Functional Requirements

**REQ-SW-7:** The library **MUST** provide an operation that accepts a username and a password and returns a result indicating whether the credentials match a stored record.

**REQ-SW-8:** The library **MUST** return a distinct result value for each of the following outcomes: success, credential mismatch, account locked, invalid input.

**REQ-SW-9:** The library **MUST** maintain, for each username, a counter of consecutive failed authentication attempts.

**REQ-SW-10:** The library **MUST** lock an account after three consecutive failed authentication attempts and **MUST** reject all subsequent authentication attempts on that account without consulting the credential file, until the lock is released.

**REQ-SW-11:** The library **MUST** reset the failed-attempts counter of a username to zero upon a successful authentication for that username.

**REQ-SW-12:** The library **MUST NOT** reveal, through its return values or log messages, whether an authentication failure was caused by an unknown username or by a wrong password, to limit information leakage to potential attackers.

### 4.3 User Account Removal

**Objective** — Allow an administrator to remove an existing user record from the library. Source: REQ1 of the running project and PBI-03 from Hands-on #1.

**Actors** — Administrator (primary, active).

#### Flow of Events

*Basic Flow.* The administrator invokes the deletion operation with a target username. The library reads the credential file line by line, copies every record that does not belong to the target user into a new in-memory list, and rewrites the credential file from that list. The operation returns success.

*Alternative Flow(s).* If the target username is null or empty, the operation returns a failure code without touching the file. If the target user is not found in the file after the full scan, the operation returns a failure code and leaves the file unchanged.

#### 4.3.1 Description and Priority

Account removal is a low frequency but essential administrative operation.

**Priority: High** because it is explicitly named in REQ1 of the running project.

#### 4.3.2 Stimulus/Response Sequences

The administrator provides a username. The system responds with success when the record has been removed, or with a failure code when the username is invalid or does not exist.

#### 4.3.3 Functional Requirements

**REQ-SW-13:** The library **MUST** provide an operation that accepts a username and removes the corresponding record from the credential file.

**REQ-SW-14:** The library **MUST** reject any deletion attempt in which the username is null or empty.

**REQ-SW-15:** The library **MUST** return a failure code when the requested username does not exist in the credential file.

**REQ-SW-16:** On successful deletion, the library **MUST** preserve all other records in the credential file unchanged.

### 4.4 Password Management

**Objective** — Allow a user to change the password associated with an existing account. Source: REQ1 of the running project (the "modifying users" clause) and PBI-04 from Hands-on #1.

**Actors** — User (primary, active). Administrator inherits this capability through actor generalization.

#### Flow of Events

*Basic Flow.* The user invokes the password-change operation with their username and a new password. The library reads the credential file line by line, replaces the password of the matching record, and rewrites the file. The operation returns success.

*Alternative Flow(s).* If the username or the new password is null, or if the new password is empty, the operation returns a failure code without touching the file. If the username is not found in the file, the operation returns a failure code and leaves the file unchanged.

#### 4.4.1 Description and Priority

Password management allows users to keep their credentials current and to react if they suspect their password has been compromised. **Priority: Medium**, since the account remains usable without invoking this feature, but its absence would force administrative intervention for every password update.

#### 4.4.2 Stimulus/Response Sequences

The user provides a username and a new password. The system responds with success when the record has been updated, or with a failure code when the input is invalid or the username does not exist.

#### 4.4.3 Functional Requirements

**REQ-SW-17:** The library **MUST** provide an operation that accepts a username and a new password and replaces the password of the matching record in the credential file.

**REQ-SW-18:** The library **MUST** reject any password-change attempt in which the username is null, the new password is null, or the new password is empty.

**REQ-SW-19:** The library **MUST** return a failure code when the requested username does not exist in the credential file.

**REQ-SW-20:** On a successful password change, the library **MUST** preserve the username of the record unchanged and **MUST** preserve all other records in the credential file unchanged.

## 5. Other Nonfunctional Requirements

### 5.1 Performance Requirements

The Authentication Library targets the modest performance expectations of an academic exercise and not those of a production-grade authentication backend.

### 5.2 Safety Requirements

Not applicable.

### 5.3 Security Requirements

Security is the dominant non-functional concern of the library, even though several stronger protections have been deliberately left out of scope. The requirements below describe the minimum-security behavior that the current version **MUST** exhibit.

**REQ-NF-1:** The library **MUST** lock any account after three consecutive failed authentication attempts and **MUST** reject further authentication attempts on that account until the lock is released. (Traces to REQ-SW-10.)

**REQ-NF-2:** The library **MUST NOT** disclose, through return values or log output, whether an authentication failure is due to an unknown username or to a wrong password. (Traces to REQ-SW-12.)

### 5.4 Software Quality Attributes

- Maintainability
- Portability

- Reusability
- Testability

## 5.5 Business Rules

Only an Administrator may invoke user registration (`authAdd`) and user deletion (`authDelete`). End users may invoke authentication and password change for their own account.

## 6. Other Requirements

No additional requirements apply.

## 7. Traceability matrix

| Requirements from upper layer document<br>Hands-On 1                                                   | This SRS                                                                                                                                                                                          |
|--------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>REQ1</i> — The Authentication Library must allow registering, modifying, removing users             | <i>REQ-SW-1, REQ-SW-2, REQ-SW-3, REQ-SW-4, REQ-SW-5, REQ-SW-6 (registration); REQ-SW-13, REQ-SW-14, REQ-SW-15, REQ-SW-16 (removal); REQ-SW-17, REQ-SW-18, REQ-SW-19, REQ-SW-20 (modification)</i> |
| <i>REQ2</i> — The Authentication Library must verify that a user is requesting a legit authentication. | <i>REQ-SW-7, REQ-SW-8, REQ-SW-9, REQ-SW-10, REQ-SW-11, REQ-SW-12; REQ-NF-1, REQ-NF-2</i>                                                                                                          |
|                                                                                                        |                                                                                                                                                                                                   |

## Appendix A: Glossary

**Administrator** — A privileged operator authorized to register and delete user records.

**Authentication** — The act of verifying that a presented username/password pair matches a previously stored record.

**Credential file** — The flat text file used by the library to persist user records; one record per line in the form `username:password`.

**End User** — A person whose credentials are managed by the library and who interacts with the library through a host application.

**Lockout** — The state in which an account is temporarily refused authentication, triggered by three consecutive failed attempts.

**PBI** — Product Backlog Item, the unit of work tracked during Hands-on #1.

**SRS** — Software Requirements Specification, the present document.

## **Appendix B: Analysis Models**

The use case diagram included in Section 4 is the only analysis model produced for this SRS.

## **Appendix C: To Be Determined List**

*Not applicable.*